

BÖLÜM 4

Sözcüksel ve Sözdizimsel Analiz

İçindekiler

- 4.1 Giriş
- 4.2 Sözcüksel Analiz (Lexical Analysis)
- 4.3 Ayrıştırma Problemi (The Parsing Problem)
- 4.4 Özyinelemeli İniş Ayrıştırma (Recursive-Descent Parsing)
- 4.5 Aşağıdan Yukarı Ayrıştırma (Bottom-Up Parsing)
- Sınav Soruları ve Cevapları
- Kitap Sonu Soruları ve Çözümleri

Kaynak: Concepts of Programming Languages, 12. Baskı Robert W. Sebesta

4.1 Giriş (Introduction)

Bir derleyicinin (compiler) tasarımı, en az bir sömestr yoğun çalışma gerektiren ciddi bir mühendislik disiplini. Bu çalışmanın ilk bölümü sözcüksel ve sözdizimsel analizlere ayrılır. **Sözdizim çözümleyici (syntax analyzer / parser)**, bir derleyicinin kalbidir; çünkü semantik çözümleyici ve ara kod üreticisi gibi önemli bileşenler onun eylemlerine göre çalışır.

■ Neden Bu Konu Önemli?

Sözdizimsel analiz yalnızca derleyici tasarımına özgü değildir. Kaynak kodu biçimlendirme araçları, program karmaşıklığı hesaplayıcıları ve yapılandırma dosyası okuyucuları gibi pek çok yazılım da sözcüksel ve sözdizimsel analiz yapar.

Üç Temel Uygulama Yaklaşımı

Yaklaşım	Açıklama	Örnek Kullanım	Analiz Gerektirir mi?
Derleme (Compilation)	Yüksek seviye dili makine koduna çevirir	C++, COBOL	Evet
Saf Yorumlama (Pure Interpretation)	Çeviri olmadan doğrudan yürütür	JavaScript (eski tarayıcı)	Evet
Hibrit (Hybrid)	Ara forma çevirir, yorumlar	Java (JVM), .NET	Evet

■ JIT Derleyici

Just-in-Time (JIT) derleyici, hibrit sistemleri gecikmeli derleme sistemine dönüştürür. Metotlar ilk çağrıldıklarında ara koddan makine koduna derlenir. Bu sayede yorumlama yavaşlığı büyük ölçüde giderilir.

BNF Kullanımının Avantajları

- Netlik ve özlük:** Hem insanlar hem de yazılım sistemleri için açık ve anlaşılırdır.
- Doğrudan temel oluşturma:** BNF açıklaması, sözdizim çözümleyicinin temeli olarak kullanılır.
- Bakım kolaylığı:** Modüler yapısı sayesinde bakımı görece kolaydır.

4.2 Sözcüksel Analiz (Lexical Analysis)

Sözcüksel çözümleyici (lexical analyzer / lexer / scanner), temel olarak bir **örüntü eşleştiricidir (pattern matcher)**. Girdi karakter dizisini tarar; karakterleri anlamlı gruplara —**sözcük birimlerine (lexeme)**— ayırır ve her gruba bir **belirteç (token)** kodu atar. Sözdizim çözümleyicisinin yegane girdi kaynağı, sözcüksel çözümleyicinin ürettiği bu belirteç akışıdır.

Temel Kavramlar

Kavram	Tanım	Örnek
Lexeme (Sözcük Birimi)	Girdi karakter dizisindeki en küçük anlamlı birim	result, =, 47, ;
Token (Belirteç)	Lexeme kategorisini temsil eden sayısal kod	IDENT=11, INT_LIT=10
Pattern (Örüntü)	Lexeme'leri tanımlayan kural/kalıp	Harf ile başlayan alfanümerik dizi

Örnek: Atama İfadesinin Sözcüksel Analizi

Aşağıdaki C ifadesi sözcüksel analize tabi tutulduğunda:

```
result = oldsum - value / 100;
```

Token	Lexeme	Kod
IDENT	result	11
ASSIGN_OP	=	20
IDENT	oldsum	11
SUB_OP	-	22
IDENT	value	11
DIV_OP	/	24
INT_LIT	100	10
SEMICOLON	;	—

Sözcüksel Analizin Sözdizimsel Analizden Ayrılmasının Nedenleri

Neden	Açıklama
Basitlik (Simplicity)	Sözcüksel analiz teknikleri daha az karmaşıktır; ayrı tutulması her iki bileşeni de sade kılar.
Verimlilik (Efficiency)	Toplam derleme süresinin önemli kısmını oluşturduğundan, sözcüksel çözümleyici ayrıca optimize edilebilir.
Taşınabilirlik (Portability)	Sözcüksel çözümleyici platforma bağlı I/O işlemleri içerir; sözdizim çözümleyici ise platform bağımsız kalabilir.

Sözcüksel Çözümleyici Tasarlama Yaklaşımları

- **1. Araç tabanlı:** Düzenli ifade benzeri açıklamalar (lex, flex) → otomatik üretim.
- **2. Durum geçiş diyagramı + kod:** Diyagram elle çizilir, kod yazılır.
- **3. Durum geçiş diyagramı + tablo:** Diyagram elle çizilir, tablo sürümlü uygulama yazılır.

Durum Geçiş Diyagramı (State Transition Diagram)

Bir **durum geiş diyagramı (state diagram)**, yönlü bir çizgedir (directed graph). Düğümler durumları (state), yaylar ise geişi tetikleyen girdi karakterlerini temsil eder. Bu diyagramlar matematiksel olarak **sonlu özdevinim (finite automaton)** sınıfına girer. Belirtelerin dili bir **düzenli dil (regular language)**, sözcüksel çözümleyici ise bir sonlu özdevinim olarak modellenebilir.

Karakter Sınıfları (Character Classes)

Her karakteri ayrı ayrı işlemek yerine sınıflar tanımlanır; bu durum diyagramı önemli ölçüde sadeleştirir:

- **LETTER**: Tüm büyük ve küçük harfler (52 karakter → tek geiş)
- **DIGIT**: 0-9 rakamları (10 karakter → tek geiş)
- **UNKNOWN**: Diğerk tüm karakterler (operatörler, parantezler vb.)

Temel Yardımcı İşlevler

İşlev	Görevi
getChar()	Sonraki girdi karakterini okur; nextChar ve charClass'ı günceller.
addChar()	nextChar'ı lexeme dizisine ekler.
getNonBlank()	Boşluk olmayan bir karakter bulana kadar getChar() çağırır.
lookup()	Tek karakterli belirteleri (operatörler, parantezler) tanır ve kodlarını döndürür.
lex()	Ana sözcüksel çözümleyici; bir sonraki lexeme ve tokenı bulup döndürür.

C ile Gerçeklenmiş Sözcüksel Çözümleyiciden Örnekler

```
/* Karakter sınıfı sabitleri */
#define LETTER 0
#define DIGIT 1
#define UNKNOWN 99

/* Belirte kodları */
#define INT_LIT 10 /* Tamsayı sabiti */
#define IDENT 11 /* Tanımlayıcı */
#define ASSIGN_OP 20 /* = */
#define ADD_OP 21 /* + */
#define SUB_OP 22 /* - */
#define MULT_OP 23 /* * */
#define DIV_OP 24 /* / */
#define LEFT_PAREN 25 /* ( */
#define RIGHT_PAREN 26 /* ) */
```

```
/* lex() - Ana sözcüksel çözümleyici */
int lex() {
    lexLen = 0;
    getNonBlank();
```

```

switch (charClass) {
    case LETTER:          /* Tanımlayıcı analizi */
        addChar(); getChar();
        while (charClass == LETTER || charClass == DIGIT) {
            addChar(); getChar();
        }
        nextToken = IDENT;
        break;
    case DIGIT:           /* Tamsayı sabiti analizi */
        addChar(); getChar();
        while (charClass == DIGIT) {
            addChar(); getChar();
        }
        nextToken = INT_LIT;
        break;
    case UNKNOWN:         /* Operatör / parantez */
        lookup(nextChar);
        getChar();
        break;
    case EOF:
        nextToken = EOF;
        break;
}
return nextToken;
}

```

Örnek Çıktı: (sum + 47) / total ifadesi

```

Next token is: 25   Next lexeme is (
Next token is: 11   Next lexeme is sum
Next token is: 21   Next lexeme is +
Next token is: 10   Next lexeme is 47
Next token is: 26   Next lexeme is )
Next token is: 24   Next lexeme is /
Next token is: 11   Next lexeme is total
Next token is: -1   Next lexeme is EOF

```

⚠ Ayrılmış Sözcükler (Reserved Words)

Ayrılmış sözcükler (if, while, for vb.) ile tanımlayıcılar aynı örüntüyle tanınır. Önce IDENT belirteci atanır; ardından ayrılmış sözcükler tablosunda arama yapılarak söz konusu lexeme'in ayrılmış sözcük olup olmadığı belirlenir.

□ Sembol Tablosu (Symbol Table)

Sözcüksel çözümleyici, kullanıcı tanımlı isimleri sembol tablosuna ilk ekleyen bileşendir. Değişken türleri gibi öznitelikler ise daha sonraki aşamalarda tabloya yazılır.

4.3 Ayırıştırma Problemi (The Parsing Problem)

Sözdizimsel analiz süreci "ayırıştırma (parsing)" olarak da anılır; bu iki terim birbirinin yerine kullanılabilir. Ayırıştırıcılar (parsers), verilen programlar için **ayırıştırma ağaçları (parse trees)** oluşturur. Bazen ağaç yalnızca örtülü biçimde kurulur; yani yalnızca ağacın dolaşımı üretilir.

Sözdizimsel Analizin İki Hedefi

- **1. Hata tespiti ve kurtarma:** Girdi program sözdizimsel olarak doğru mu? Değilse, tanı mesajı üretilmeli ve analiz devam edebilmek için kurtarılmalıdır.
- **2. Ayırıştırma ağacı üretimi:** Sözdizimsel olarak doğru girdi için tam ayırıştırma ağacı veya onun izini (trace) üretilir; bu iz çeviri için temel alınır.

Notasyon Kuralları

Sembol	Temsil Ettiği
a, b, ...	Terminal semboller (küçük harf, alfabenin başı)
A, B, ...	Terminal olmayan semboller (büyük harf, alfabenin başı)
W, X, Y, Z	Terminal veya terminal olmayan semboller (büyük harf, sonu)
w, x, y, z	Terminal dizileri (küçük harf, sonu)
$\alpha, \beta, \delta, \gamma$	Karışık diziler — terminaller ve/veya terminal olmayanlar

Yukarıdan Aşağı Ayırıştırıcılar (Top-Down Parsers)

Bir yukarıdan aşağı ayırıştırıcı, ayırıştırma ağacını **kökten yapraklara doğru** kurar. Bu, **en sol türetmeye (leftmost derivation)** karşılık gelir. Mevcut sentential form $xA\alpha$ olduğunda, A soldan en soldaki terminal olmayan semboldür; doğru kuralı seçerek sonraki sentential formu bulmak **ayırıştırma karar sorununu** oluşturur.

En yaygın yukarıdan aşağı algoritmalar **LL** ailesidir: ilk L soldan sağa tarama, ikinci L en sol türetmeyi belirtir.

Aşağıdan Yukarı Ayırıştırıcılar (Bottom-Up Parsers)

Bir aşağıdan yukarı ayırıştırıcı, ağacı **yapraklardan köke doğru** kurar. Bu, **en sağ türetmenin tersi (reverse of rightmost derivation)**ne karşılık gelir. En yaygın aşağıdan yukarı algoritmalar **LR** ailesidir: L soldan sağa tarama, R en sağ türetmeyi belirtir.

Özellik	Yukarıdan Aşağı (LL)	Aşağıdan Yukarı (LR)
Ağaç Oluşturma	Kökten yapraklara	Yapraklardan köke
Türetme Türü	En sol (Leftmost)	En sağın tersi (Reverse Rightmost)
Sol Özyineleme	Sorun — yasak	Sorun değil

Güç	LL gramerler	LR gramerler (LL üst kümesi)
Uygulama Örneği	Recursive-Descent	LR / LALR Parser

Ayrıştırmanın Karmaşıklığı (Complexity of Parsing)

□ $O(n^3)$ Genel Gramer Ayrıştırıcıları

Herhangi bir belirsiz olmayan gramere çalışan algoritmalar $O(n^3)$ karmaşıklığa sahiptir; çok sayıda geri izleme (backtracking) gerektirir. Bu derleyiciler için pratikte kullanışsızdır.

□ $O(n)$ Programlama Dili Ayrıştırıcıları

Ticari derleyicilerdeki tüm ayrıştırma algoritmaları $O(n)$ karmaşıklığa sahiptir. Bunlar, tüm gramerlerin yalnızca bir alt kümesi üzerinde çalışır; ancak bu alt küme, programlama dillerini tanımlamaya yeterlidir.

4.4 Özyinelemeli İniş Ayrıştırma (Recursive-Descent Parsing)

Bir **özyinelemeli iniş ayrıştırıcısı (recursive-descent parser)**, dilinin BNF/EBNF tanımından doğrudan yazılan bir program koleksiyonundan oluşur. Her terminal olmayan (nonterminal) sembol için bir alt program bulunur; bu alt program, söz konusu sembolün ürettiği dilin ayrıştırıcısı görevini üstlenir.

□ EBNF ve Özyinelemeli İniş

EBNF, özyinelemeli iniş ayrıştırıcıları için idealdir. Küme parantezleri $\{ \}$ "sıfır veya daha fazla kez tekrar" anlamına gelir; köşeli parantezler $[]$ ise isteğe bağlı (bir kez veya hiç) yapıyı gösterir.

Örnek EBNF Grameri — Aritmetik İfadeler

```
<expr>  -> <term> { (+ | -) <term> }
<term>  -> <factor> { (* | /) <factor> }
<factor> -> id | int_constant | ( <expr> )
```

expr() Fonksiyonu

```
/* <expr> -> <term> { (+ | -) <term> } */
void expr() {
    printf("Enter <expr>\n");
    term(); /* İlk terimi ayristir */
    /* Sonraki belirtec + veya - ise devam et */
    while (nextToken == ADD_OP || nextToken == SUB_OP) {
        lex();
        term();
    }
}
```

```
    printf("Exit <expr>\n");  
}
```

term() Fonksiyonu

```
/* <term> -> <factor> {(* | /) <factor>} */  
void term() {  
    printf("Enter <term>\n");  
    factor(); /* İlk faktörü ayrıştır */  
    while (nextToken == MULT_OP || nextToken == DIV_OP) {  
        lex();  
        factor();  
    }  
    printf("Exit <term>\n");  
}
```

factor() Fonksiyonu — Çok Sayıda Sağ Taraf

```
/* <factor> -> id | int_constant | ( <expr> ) */  
void factor() {  
    printf("Enter <factor>\n");  
    if (nextToken == IDENT || nextToken == INT_LIT)  
        lex(); /* id veya int_constant: sonrakine geç */  
    else if (nextToken == LEFT_PAREN) {  
        lex(); /* ( yi geç */  
        expr(); /* iç ifadeyi ayrıştır */  
        if (nextToken == RIGHT_PAREN)  
            lex();  
        else  
            error();  
    }  
    else error();  
    printf("Exit <factor>\n");  
}
```

İz Örneği: (sum + 47) / total

```
Giris belirteci: 25   Lexeme: (  
Enter <expr>  
  Enter <term>  
    Enter <factor>      ← ( görüldü, iç ifadeye gir  
      Enter <expr>  
        Enter <term>  
          Enter <factor> ← sum (IDENT)  
            Exit <factor>  
          Exit <term>
```



```

    (+ goruldu)
    Enter <term>
        Enter <factor> ← 47 (INT_LIT)
        Exit <factor>
    Exit <term>
Exit <expr>
) goruldu - Exit <factor>
Exit <term>
(/ goruldu)
Enter <term>
    Enter <factor> ← total (IDENT)
    Exit <factor>
Exit <term>
Exit <expr>

```

4.4.2 LL Gramer Sınıfı — Kısıtlamalar

Sorun 1: Sol Özyineleme (Left Recursion)

□ Sol Özyineleme Nedir?

$A \rightarrow A + B$ gibi bir kuralda A için yazılan alt program, hemen kendini çağırır $\rightarrow$ sonsuz döngü $\rightarrow$ yığın taşması.

Doğrudan sol özyinelemeyi giderme süreci:

```

/* Orijinal (Sol Özyinelemeli - SORUNLU) */
E -> E + T | T
T -> T * F | F
F -> (E) | id

/* Donusturulmus (Sol Özyinelemesiz - DOGRU) */
E -> T E'
E' -> + T E' | epsilon
T -> F T'
T' -> * F T' | epsilon
F -> (E) | id

```

Sorun 2: İkili Ayırıklık Testi (Pairwise Disjointness Test)

Bir terminal olmayan A için birden fazla sağ taraf (RHS) varsa, her çift RHS'nin başlangıç kümeleri —**FIRST kümeleri**— ayırık (disjoint) olmalıdır. Aksi hâlde ayrıştırıcı hangi RHS'yi kullanacağını bilemez.

FIRST(α)	Tanım
FIRST(α)	α 'dan sıfır veya daha fazla adımda türetilebilen, $a \rightarrow \alpha\beta$ şeklinde elde edilen tüm terminal a'ların kümesi. $\alpha \rightarrow^* \epsilon$ ise ϵ de bu kümededir.

İkili ayırıklık testi için örnek:

```

/* GECEN kural – FIRST kumeleri ayrik */
A -> aB | bAb | Bb
B -> cB | d
FIRST(aB) = {a}
FIRST(bAb) = {b}
FIRST(Bb) = {c, d}
=> Kesisimlerin hepsi bos – TEST GECTI

/* GECEMEYEN kural – FIRST kumeleri ortusur */
A -> aB | BAb
B -> aA | b
FIRST(aB) = {a}
FIRST(BAb) = {a, b}
FIRST(aB) n FIRST(BAb) = {a} ≠ {} – TEST KALDI

```

Sol Çarpanlama (Left Factoring)

İkili ayrıklık testini geçemeyen grameri düzeltmek için kullanılan bir tekniktir. Ortak önek faktör olarak çıkarılır.

```

/* Orijinal (test kalamayan) */
<variable> -> identifier | identifier [<expression>]

/* Sol carpanlama sonrasi */
<variable> -> identifier <new>
<new>      -> epsilon | [<expression>]

/* Veya EBNF ile (daha sade) */
<variable> -> identifier [[<expression>]]
      (dis kose parantezler meta sembol = ihtiyari)
      (ic kose parantezler programlama dilinin terminali)

```

4.5 Aşağıdan Yukarı Ayırıştırma (Bottom-Up Parsing)

Aşağıdan yukarı ayırıştırmada temel görev, mevcut sağ sentential formda **tutamacı (handle)** bulup onu karşılık gelen sol tarafa (LHS) indirgemektir (reduce). Bu süreç, en sağ türetmenin tersidir.

Önemli Kavramlar

Kavram	Tanım
Sağ Sentential Form	En sağ türetmede görünen sentential form.
Phrase (Öbek)	Tüm ayırıştırma ağacındaki tek bir iç düğümde köklenen alt ağacın tüm yapraklarından oluşan dize. (1+ türetme adımı)
Simple Phrase (Basit Öbek)	Tam olarak bir türetme adımıyla elde edilen öbek; daima bir RHS'dir.
Handle (Tutamac)	Bir sağ sentential formun en soldaki basit öbeği. Benzersizdir; indirgeme işlemi burada yapılır.

Örnek: Tutamacı Bulma

```
Gramer:  
S -> aAc  
A -> aA | b  
  
Türetme: S => aAc => aaAc => aabc  
  
Ayrıştırma (alt-üst) aabc'den başla:  
Sentential form: aabc  
İçerik: a a b c  
Tek RHS: b (A -> b)  
Handle = b → A ile değiştir => aaAc  
Handle = aA (A -> aA) => aAc  
Handle = aAc (S -> aAc) => S
```

4.5.2 Kaydır-İndirge Algoritmaları (Shift-Reduce)

Aşağıdan yukarı ayrıştırıcılar çoğunlukla kaydır-indirge (shift-reduce) olarak adlandırılır. Bir **yığın (stack)** kullanılır; iki temel eylem vardır:

- **Shift (Kaydır):** Sonraki girdi belirtecini yığına iter.
- **Reduce (İndirge):** Yığının tepesindeki tutamacı (handle) karşılık gelen LHS ile değiştir.
- **Accept:** Ayrıştırma başarıyla tamamlandı.
- **Error:** Sözdizimsel hata tespit edildi.

□ Pushdown Automaton (PDA)

Her programlama dili ayrıştırıcısı bir PDA'dır; çünkü PDA, bağlamsız dillerin (context-free language) tanıyıcısıdır. Özyinelemeli iniş ayrıştırıcısı da bir PDA'dır — yığını çalışma zamanı sistemi sağlar.

4.5.3 LR Ayrıştırıcılar

LR algoritması Donald Knuth tarafından 1965'te tasarlanmıştır. Sonradan daha verimli varyantlar geliştirilmiştir (SLR, LALR, Canonical LR). Tüm LR ayrıştırıcılar aynı ayrıştırma algoritmasını kullanır; yalnızca ayrıştırma tabloları farklıdır.

LR Ayrıştırıcılarının Avantajları

- **1. Kapsam:** Tüm programlama dilleri için LR ayrıştırıcı üretilebilir.
- **2. Erken hata tespiti:** Soldan sağa taramada hatalar mümkün olan en erken noktada tespit edilir.
- **3. Güç:** LR gramerler sınıfı, LL gramerler sınıfının katı bir üst kümesidir (birçok sol özyinelemeli gramer LR'dir, LL değildir).

⚠ Yegane Dezavantaj

LR ayrıştırma tablosunu elle üretmek son derece güçtür. Ancak yacc, bison, ANTLR gibi araçlar grameri girdi olarak alarak tabloyu otomatik üretir.

LR Ayrıştırma Tablosu Yapısı

Bölüm	Satır Etiketi	Sütun Etiketi	İçerik
ACTION	Durum sembolü	Terminal semboller	Shift N / Reduce N / Accept / Error
GOTO	Durum sembolü	Terminal olmayanlar	Hangi durum sembolü yığına itileceği

Örnek Gramer (Aritmetik İfadeler)

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow id$

LR Ayrıştırma İzi: $id + id * id$

Yığın (Stack)	Girdi	Eylem
0	$id + id * id \$$	Shift 5
0 id 5	$+ id * id \$$	Reduce 6 $\rightarrow$ F (GOTO[0,F]=3)
0 F 3	$+ id * id \$$	Reduce 4 $\rightarrow$ T (GOTO[0,T]=2)
0 T 2	$+ id * id \$$	Reduce 2 $\rightarrow$ E (GOTO[0,E]=1)
0 E 1	$+ id * id \$$	Shift 6
0 E 1 + 6	$id * id \$$	Shift 5
0 E 1 + 6 id 5	$* id \$$	Reduce 6 $\rightarrow$ F (GOTO[6,F]=3)
0 E 1 + 6 F 3	$* id \$$	Reduce 4 $\rightarrow$ T (GOTO[6,T]=9)
0 E 1 + 6 T 9	$* id \$$	Shift 7
0 E 1 + 6 T 9 * 7	$id \$$	Shift 5
0 E 1 + 6 T 9 * 7 id 5	$\$$	Reduce 6 $\rightarrow$ F (GOTO[7,F]=10)
0 E 1 + 6 T 9 * 7 F 10	$\$$	Reduce 3 $\rightarrow$ T (GOTO[6,T]=9)
0 E 1 + 6 T 9	$\$$	Reduce 1 $\rightarrow$ E (GOTO[0,E]=1)
0 E 1	$\$$	ACCEPT

□ Knuth'un İçgörüsü

Ayrıştırma yığınının tüm geçmişi tek bir "durum" sembolüyle temsil edilebilir. Yığının tepesindeki durum sembolü, mevcut ana kadar gerçekleştirilen tüm ayrıştırma kararlarının özeti niteliğindedir.

Bölüm Özeti

Konu	Temel Nokta
Sözcüksel Analiz	Pattern matcher; karakter → lexeme → token dönüşümü.
Ayrıştırma	Parse tree üretimi; iki hedef: hata tespiti ve ağaç oluşturma.
Top-Down (LL)	Kökten yapraklara; sol özyineleme yasak; FIRST kümesi kontrolü.
Bottom-Up (LR)	Yapraklardan köke; sol özyinelemeli gramerleri destekler.
Recursive-Descent	BNF/EBNF'den doğrudan yazılan LL ayrıştırıcı.
LR Ayrıştırıcı	Shift-Reduce; ACTION+GOTO tabloları; $O(n)$ karmaşıklık.
Karmaşıklık	Genel: $O(n^3)$. Programlama dilleri için: $O(n)$.

Olası Sınav Soruları ve Cevapları

S1. Sözcüksel analiz (lexical analysis) ile sözdizimsel analiz (syntax analysis) arasındaki temel farkları açıklayınız. Ayrılımlarının üç gerekçesini belirtiniz.

Cevap:

Sözcüksel analiz, programın küçük ölçekli yapılarıyla (isimler, sayı sabitleri, operatörler) ilgilenir ve karakterleri lexeme'lere gruplayarak token kodlarını üretir. Sözdizimsel analiz ise büyük ölçekli yapıları (ifadeler, deyimler, program birimleri) inceler ve ayrıştırma ağacını kurar. Ayrılma nedenleri: (1) Basitlik — sözcüksel analiz teknikleri daha az karmaşık; her iki bileşen daha sade olur. (2) Verimlilik — sözcüksel çözümleyici ayrıca optimize edilebilir. (3) Taşınabilirlik — sözcüksel çözümleyici platforma bağlı I/O içerirken, sözdizim çözümleyici platform bağımsız kalır.

S2. "Lexeme" ve "token" kavramlarını tanımlayınız. Aşağıdaki C ifadesinin her bileşeni için token ve lexeme değerlerini gösteriniz: $x = a + 3;$

Cevap:

Lexeme: Girdi karakter dizisinde en küçük anlamlı birim. Token: Bir lexeme kategorisini temsil eden sayısal/sembolik kod. Tablo:

$x \rightarrow$ IDENT (11)

$= \rightarrow$ ASSIGN_OP (20)

$a \rightarrow$ IDENT (11)

$+ \rightarrow$ ADD_OP (21)

$3 \rightarrow$ INT_LIT (10)

$; \rightarrow$ SEMICOLON

S3. Yukarıdan aşağı ayrıştırıcılar için "sol özyineleme (left recursion)" neden sorun teşkil eder? $A \rightarrow A + B$ kuralından hareketle açıklayınız ve doğrudan sol özyinelemeyi giderme sürecini gösteriniz.

Cevap:

$A \rightarrow A + B$ kuralında A için yazılan özyinelemeli iniş alt programı, çalışır çalışmaz hemen A'yı tekrar çağırır. Bu durum, hiçbir girdi tüketilmeden sonsuz özyinelemeye (infinite recursion) ve dolayısıyla yığın taşmasına (stack overflow) yol açar. Giderme: $A \rightarrow A + B \mid B$ biçimindeki kural için $\alpha_1 = "+B"$ ve $\beta = "B"$ alınır. Yeni kurallar: $A \rightarrow B A'$ ve $A' \rightarrow +B A' \mid \epsilon$

S4. FIRST kümesini tanımlayınız. İkili ayrıklık testini (pairwise disjointness test) açıklayınız ve aşağıdaki kurallar için testi uygulayınız: $A \rightarrow aB \mid bAb \mid Bb B \rightarrow cB \mid d$

Cevap:

$FIRST(\alpha) = \{a \mid \alpha \rightarrow^* a\beta\} \cup \{\epsilon, \text{eğer } \alpha \rightarrow^* \epsilon\}$; yani α 'dan türetilebilen tüm baştaki terminaller. Pairwise Disjointness Testi: A'nın her çift RHS'si için FIRST kümelerinin kesişimi boş olmalıdır. Uygulama:

$FIRST(aB) = \{a\}$

$FIRST(bAb) = \{b\}$

$FIRST(Bb) = FIRST(B) = \{c, d\}$

Tüm çift kesişimler boş $\rightarrow$ Test geçildi $\rightarrow$ Bu kural seti için özyinelemeli iniş ayrıştırıcısı yazılabilir.

S5. LR ayrıştırıcılarının üç avantajını belirtiniz. ACTION ve GOTO tablolarının rollerini açıklayınız.

Cevap:

1. Tüm programlama dilleri için LR ayrıştırıcı üretilebilir. 2. Soldan sağa taramada hatalar en erken noktada saptanır. 3. LR gramer sınıfı, LL gramer sınıfının katı üst kümesidir. ACTION tablosu: Satır = mevcut durum, Sütun = sonraki terminal belirteç. Hücre değeri: Shift N (yığına it, durum N'ye geç), Reduce N (kural N ile indirge), Accept veya Error. GOTO tablosu: Bir indirgeme tamamlandıktan sonra yığına hangi durum sembolünün itileceğini belirtir. Satır = indirgeme sonrasında açığa çıkan durum, Sütun = indirgemedede kullanılan kuralın LHS nonterminal'i.

S6. "Handle (tutamac)" kavramını tanımlayınız ve aşağıdaki gramer ile türetme için her sentential formun tutamacını gösteriniz: Gramer: $E \rightarrow E+T \mid T$, $T \rightarrow T^*F \mid F$, $F \rightarrow id \mid id + id * id$

Cevap:

Handle: Bir sağ sentential formun en soldaki basit öbeğidir. Benzersizdir ve indirgemenin gerçekleştirileceği yerdir. Adımlar:

1. $id + id * id \rightarrow$ handle: $id (F \rightarrow id)$, indirgeme: F
2. $F + id * id \rightarrow$ handle: $F (T \rightarrow F)$, indirgeme: T
3. $T + id * id \rightarrow$ handle: $T (E \rightarrow T)$, indirgeme: E
4. $E + id * id \rightarrow$ handle: $id (F \rightarrow id)$, indirgeme: F
5. $E + F * id \rightarrow$ handle: $F (T \rightarrow F)$, indirgeme: T
6. $E + T * id \rightarrow$ handle: $id (F \rightarrow id)$, indirgeme: F
7. $E + T * F \rightarrow$ handle: $T^*F (T \rightarrow T^*F)$, indirgeme: T
8. $E + T \rightarrow$ handle: $E+T (E \rightarrow E+T)$, indirgeme: E
9. $E \rightarrow$ KABUL

S7. Özyinelemeli iniş ayrıştırıcısı nedir? Her alt programın sorumluluğunu ve tek RHS'li bir kural için alt programın nasıl yazıldığını açıklayınız.

Cevap:

Özyinelemeli iniş ayrıştırıcısı, dilin BNF/EBNF tanımından doğrudan yazılan, her terminal olmayan için bir alt program içeren ve yukarıdan aşağı ayrıştırma ağacı üreten bir ayrıştırıcıdır. Her alt program, ilgili terminal olmayanın ürettiği dizinin ayrıştırıcısıdır. Tek RHS'li kural için: RHS soldan sağa taranır; her terminal sembol nextToken ile karşılaştırılır (eşleşirse lex() çağrılır, eşleşmezse hata verilir); her terminal olmayan için karşılık gelen alt program çağrılır. Çağrı sonrasında nextToken, kullanılmamış bir sonraki belirteci içerir.

Kitap Sonu Soruları ve Çözümleri (Review Questions)

Aşağıda kitabın ilgili bölüm sonu sorularından seçilmiş olanlar ve tam çözümleri yer almaktadır.

RQ1. Sözdizim çözümleyicilerinin (syntax analyzers) gramerlere dayandırılmasının üç nedenini açıklayınız. (Soru 1)

Çözüm:

1. BNF açıklamaları hem insanlar hem de yazılım sistemleri için açık ve özeldir. 2. BNF açıklaması sözdizim çözümleyicisinin doğrudan temeli olarak kullanılabilir. 3. BNF tabanlı uygulamalar modüler yapıları sayesinde görece kolay bakım yapılabilir.

RQ2. "Durum geçiş diyagramı (state transition diagram)" nedir? (Soru 6)

Çözüm:

Durum geçiş diyagramı yönlü bir çizgedir (directed graph). Düğümler durumları temsil eder. Yaylar, durumlar arasındaki geçişlere yol açan girdi karakterleri ile geçiş sırasında gerçekleştirilecek eylemleri içerir. Bu diyagramlar, finite automata adı verilen matematiksel makinelerin gösterimidir.

RQ3. Sözdizimsel analizin iki hedefi nedir? (Soru 8)

Çözüm:

1. Girdi programı sözdizimsel olarak denetlemek; hata bulunduğunda tanı mesajı üretip ayrıştırmayı sürdürebilmek için kurtarma yapmak. 2. Sözdizimsel olarak doğru girdi için tam ayrıştırma ağacı (veya izini) üretmek; bu ağaç çeviri için kullanılır.

RQ4. Yukarıdan aşağı ile aşağıdan yukarı ayrıştırıcıların farklarını açıklayınız. (Soru 9)

Çözüm:

Yukarıdan aşağı (top-down): Ağaç kökten yapraklara; en sol türetme üretilir; LL algoritmaları bu gruba girer. Aşağıdan yukarı (bottom-up): Ağaç yapraklardan köke; en sağ türetmenin tersi üretilir; LR algoritmaları bu gruba girer. LR gramer sınıfı, LL gramer sınıfından daha büyüktür.

RQ5. "Tutamac (handle)" nedir? (Soru 21)

Çözüm:

Tutamac, bir sağ sentential formun en soldaki basit öbeğidir. Biçimsel tanım: $\beta, \gamma = \alpha\beta\omega$ sağ sentential formunun tutamacıdır; ancak ve ancak $S \rightarrow^* \alpha A \omega \rightarrow^* \alpha \beta \omega$ şartı sağlanıyorsa. Aşağıdan yukarı ayrıştırıcı, tutamacı yığının tepesinde bulur ve indirgeme yapar.

RQ6. LR ayrıştırıcılarının üç avantajını belirtiniz. (Soru 23)

Çözüm:

1. Tüm programlama dilleri için inşa edilebilir. 2. Hatalar soldan sağa taramada mümkün olan en erken anda saptanır. 3. LR gramer sınıfı, LL gramer sınıfının katı üst kümesidir.

RQ7. Sol özyineleme LR ayrıştırıcıları için sorun mudur? (Soru 27)

Çözüm:

Hayır. Sol özyineleme yalnızca yukarıdan aşağı (LL/recursive-descent) ayrıştırıcılar için sorun teşkil eder; çünkü özyinelemeli çağrılar hiç girdi tüketmeden sonsuz döngüye girer. LR ayrıştırıcıları, sol özyinelemeli gramerleri sorunsuz biçimde işleyebilir.

Alıştırma Çözümleri (Problem Set)

Problem 1a. İkili ayrıklık testini uygulayın: $A \rightarrow aB \mid b \mid cBB$

Çözüm:

$FIRST(aB) = \{a\}$

$FIRST(b) = \{b\}$

$FIRST(cBB) = \{c\}$

Tüm çiftlerin kesişimi boş $\rightarrow$ TEST GEÇİLDİ.

Bu kural seti için özyinelemeli iniş ayrıştırıcısı yazılabilir.

Problem 1b. İkili ayrıklık testini uygulayın: $B \rightarrow aB \mid bA \mid aBb$

Çözüm:

$FIRST(aB) = \{a\}$

$FIRST(bA) = \{b\}$

$FIRST(aBb) = \{a\}$

$FIRST(aB) \cap FIRST(aBb) = \{a\} \neq \{\} \rightarrow$ TEST KALDI.

$B \rightarrow aB$ ve $B \rightarrow aBb$ kuralları aynı baştaki terminale sahip; ayrıştırıcı hangisini kullanacağını belirleyemez.

Problem 3. $a + b * c$ dizesinin özyinelemeli iniş ayrıştırıcısı izini gösteriniz.

Çözüm:

$lex() \rightarrow nextToken = IDENT (a)$

$expr()$ çağrıldı

$term()$ çağrıldı

$factor()$ çağrıldı $\rightarrow IDENT (a) \rightarrow lex() \rightarrow nextToken = ADD_OP$

$factor()$ döndü

$term()$ döndü (MULT veya DIV yok)

ADD_OP görüldü $\rightarrow lex() \rightarrow nextToken = IDENT (b)$

$term()$ çağrıldı

$factor()$ çağrıldı $\rightarrow IDENT (b) \rightarrow lex() \rightarrow nextToken = MULT_OP$

$factor()$ döndü

$MULT_OP$ görüldü $\rightarrow lex() \rightarrow nextToken = IDENT (c)$

$factor()$ çağrıldı $\rightarrow IDENT (c) \rightarrow lex() \rightarrow nextToken = EOF$

$factor()$ döndü

$term()$ döndü

$expr()$ döndü $\rightarrow$ Ayrıştırma başarılı.

Problem 7. id * (id + id) dizesini LR tablosuyla ayrıştırınız.

Çözüm:

```
Yığın | Girdi | Eylem
0 | id*(id+id)$ | Shift 5
0 id 5 | *(id+id)$ | Reduce 6 → F; GOTO[0,F]=3
0 F 3 | *(id+id)$ | Reduce 4 → T; GOTO[0,T]=2
0 T 2 | *(id+id)$ | Shift 7
0 T 2 * 7 | (id+id)$ | Shift 4
0 T 2 * 7 ( 4 | id+id)$ | Shift 5
0 T 2 * 7 ( 4 id 5 | +id)$ | Reduce 6 → F; GOTO[4,F]=3
0 T 2 * 7 ( 4 F 3 | +id)$ | Reduce 4 → T; GOTO[4,T]=2
0 T 2 * 7 ( 4 T 2 | +id)$ | Reduce 2 → E; GOTO[4,E]=8
0 T 2 * 7 ( 4 E 8 | +id)$ | Shift 6
0 T 2 * 7 ( 4 E 8 + 6 | id)$ | Shift 5
0 T 2 * 7 ( 4 E 8 + 6 id 5 | )$ | Reduce 6 → F; GOTO[6,F]=3
0 T 2 * 7 ( 4 E 8 + 6 F 3 | )$ | Reduce 4 → T; GOTO[6,T]=9
0 T 2 * 7 ( 4 E 8 + 6 T 9 | )$ | Reduce 1 → E; GOTO[4,E]=8
0 T 2 * 7 ( 4 E 8 | )$ | Shift 11
0 T 2 * 7 ( 4 E 8 ) 11 | $ | Reduce 5 → F; GOTO[7,F]=10
0 T 2 * 7 F 10 | $ | Reduce 3 → T; GOTO[0,T]=2
0 T 2 | $ | Reduce 2 → E; GOTO[0,E]=1
0 E 1 | $ | ACCEPT
```